

Proyecto 2 - Diseño para la Gestión de Inventario en Tienda

Diego Andre Calderón Salazar - 241263

Universidad del Valle de Guatemala

Base de Datos 1

Facultad de Ingeniería - Ciencia de la Computación y Tecnologías de la información

Contexto del Negocio	2
Stack Seleccionado	3
Base de Datos	3
Frontend	3
Backend	4
Diseño de la Base de Datos y Normalización	4
Entidades Identificadas inicialmente	4
Dependencias funcionales iniciales	6
Normalización	7
Dependencias funcionales actualizadas	11
Relaciones y Cardinalidades	12
Diagrama Entidad Relación (DER)	13
Modelo Relacional	14
Formato texto	14
Diagrama	15
Anexos	15

Contexto del Negocio

La tienda se especializa en la venta de álbumes musicales en formatos físicos (CD y Vinilo), comprados a diversos proveedores. Los productos se agrupan en categorías comerciales (Novedades, Ofertas, Ediciones de Coleccionista) y géneros artísticos (Rock, Pop, Jazz). Los clientes realizan compras atendidas por empleados, donde cada transacción puede incluir múltiples álbumes en distintos formatos físicos. El sistema debe gestionar el catálogo musical (artistas, portadas y años), controlar el stock por formato y generar reportes de venta detallados.

Stack Seleccionado

Base de Datos

Se optó por MariaDB por su eficiencia en entornos de baja escala y su facilidad para gestionar credenciales específicas vía variables de entorno. Dado que el sistema no exige una robustez empresarial, este motor permite un desarrollo ágil de las funcionalidades CRUD y consultas SQL explícitas requeridas.

Frontend

Se seleccionó Vue 3 en conjunto con Vite para garantizar un entorno de desarrollo ultra rápido y una aplicación final ligera. Para la gestión de estados globales, como sesiones de usuario y persistencia del inventario, se optó por Pinia debido a su simplicidad y compatibilidad con la Composition API. La interfaz se construye sobre PrimeVue, aprovechando su robusta librería de componentes preconstruidos (tablas de datos y formularios), lo que acelera significativamente el diseño de los módulos administrativos y reportes.

Backend

Para la capa de servidor se eligió Fastify sobre Node.js, priorizando la eficiencia y el alto rendimiento en el procesamiento de peticiones hacia MariaDB. Su arquitectura basada en plugins permite un código modular y escalable, mientras que el uso nativo de JSON Schema para validación asegura que los datos de entrada cumplan con los requisitos del sistema antes de ser procesados. Esta selección permite manejar las transacciones de ventas y la lógica de inventario con una latencia mínima y gran estabilidad.

Diseño de la Base de Datos y Normalización

Entidades Identificadas inicialmente

- Producto
 - **Id**
 - Nombre
 - Precio
 - Id_categoría
 - Id_proveedor
- Categoría
 - **Id**
 - Detalle
- Proveedor
 - **Id**
 - Nombre
- Cliente (se puede sacar luego algunos atributos a una entidad persona)
 - **Id**
 - Nombre

- Nit
 - Correo (opcional)
 - Direccion (opcional)
- Empleado
 - **Id**
 - Nombre
 - DPI
- Compra (un empleado hace muchas ventas y un cliente hace muchas compras)
(una compra tiene muchos productos y un producto tiene muchas compras)
 - **Id**
 - Id_cliente
 - Id_empleado
 - Fecha (DATETIME para que guarde tambien la hora)
- Detalle-venta (tabla de cruce)
 - **Id compra**
 - **Id producto**
 - Cantidad
 - Subtotal (precio * cantidad)
- Stock
 - Id
 - Id_producto
 - Cantidad

No hice entidad de reportes ya que prefiero realizarlos mediante consultas de manera dinámica en la web y luego mostrarlos o dar la opción de imprimirlo.

Dependencias funcionales iniciales

- Producto
 - Id -> Nombre, id_proveedor, id_categoria, precio
- Categoría
 - Id -> Detalle
- Proveedor
 - Id -> nombre
- Cliente
 - Id -> Nombre, Correo, nit, dirección
 - NIT no implica nombre ni dirección debido a que el NIT puede ser CF y la dirección registrada en SAT al NIT puede ser distinta a la dirección de entrega.
- Empleado
 - Id -> Nombre, DPI
- Compra
 - Id -> id_cliente, id_empleado, fecha
- Compra-producto
 - (id_compra, id_producto) -> cantidad, subtotal
 - Id_producto -> subtotal
 - Cantidad -> subtotal
 - Rompe 2NF, pues existe una dependencia parcial donde subtotal depende únicamente del id_producto (por el cálculo de $\text{precio} * \text{cantidad}$)
- Stock
 - Id -> id_producto, cantidad

Normalización

- Producto
 - **Id**
 - Id_album (FK)
 - Id_album_tipo (FK)
 - Precio
 - Id_categoria (FK)
 - Id_proveedor (FK)
 - Stock
 - Como la relación entre la entidad stock y producto es 1:1, decidí agregar un atributo llamado stock a la entidad de producto, que registrará la cantidad de productos restantes existentes. Por este motivo eliminé la entidad Stock.
- Categoría
 - **Id**
 - Detalle
 - Esta entidad se hizo para ahorrar memoria en la base de datos y no repetir en varias tuplas los mismos strings.
- Proveedor
 - **Id**
 - Nombre
 - Esta entidad se hizo para ahorrar memoria en la base de datos y no repetir en varias tuplas los mismos strings.
- Persona
 - **Id**

- Nombre
- Correo
- Contraseña
- Id_rol (FK)
 - Agregué la entidad persona para poder tener una entidad que guardé los datos personales tanto de los empleados como de los clientes, y también para almacenar información que va a ser útil para la funcionalidad del login (como la contraseña, el correo y el rol)
- Rol
 - **Id**
 - Detalle (admin, vendedor, cliente)
 - Esta entidad se hizo para ahorrar memoria en la base de datos y no repetir en varias tuplas los mismos strings.
- Cliente
 - **Id**
 - Id_persona (FK)
 - NIT
 - Direccion (opcional)
- Empleado
 - **Id**
 - Id_persona (FK)
 - DPI
- Compra
 - **Id**

- Id_cliente (FK)
- Id_empleado (FK)
- Fecha (DATETIME para que guarde tambien la hora)
- Detalle-venta (tabla de cruce)
 - **Id compra** (FK)
 - **Id producto** (FK)
 - Cantidad
 - Precio_unitario
 - Se guarda acá además de en producto ya que el precio en ciertos momentos puede cambiar, y es importante guardar el valor del precio en ese preciso momento para luego calcular el subtotal (cantidad * precio_unitario)
 - Esto resuelve la inconsistencia que rompía la normalización de 2NF ya que ahora Precio unitario depende tanto de id_compra como de id_producto, pues es un precio estático que se extrae del producto en la fecha de realizada cierta compra. En este caso tampoco existe la dependencia de Cantidad -> subtotal debido a que ahora el subtotal se calcula fuera utilizando el precio unitario y la cantidad.

NOTA: posterior a los avances presentados, decidi que la tienda será musical, por este motivo las siguientes entidades son agregadas para coincidir con mi visión del negocio. También se modificó la entidad de producto para tener algunas FK de las nuevas entidades.

- Artista
 - **Id**

- Nombre
 - Esta entidad permite que un artista tenga muchos álbumes (1:N), evitando repetir el nombre del músico en cada disco. Además, optimiza el uso de strings en la base de datos
- Album
 - **Id**
 - Título
 - Año
 - Portada (VARCHAR para almacenar el link de la imagen)
 - Tracks
 - Id_artista (FK)
- Genero
 - **Id**
 - Detalle
- Album-Genero (Tabla de Cruce)
 - **Id album (FK)**
 - **Id genero (FK)**
 - Esto se hace porque un album puede pertenecer a varios generos y un mismo genero puede pertenecer a muchos albums.
- Album_Tipo
 - **Id**
 - Detalle (CD o vinilo)

Dependencias funcionales actualizadas

- **Artista**
 - Id -> Nombre
- **Album**
 - Id -> Titulo, anio, portada, tracks, id_artista
- **Genero**
 - Id -> detalle
- **Album_Tipo**
 - Id -> detalle
- **Categoria**
 - Id -> detalle
- **Producto**
 - Id -> id_album, id_album_tipo, id_categoria, id_proveedor, precio, stock
- **Proveedor**
 - Id -> nombre
- **Persona**
 - Id -> Nombre, correo, contraseña, id_rol
- **Rol**
 - Id -> detalle
- **Cliente**
 - Id -> id_persona, NIT, Direccion
- **Empleado**

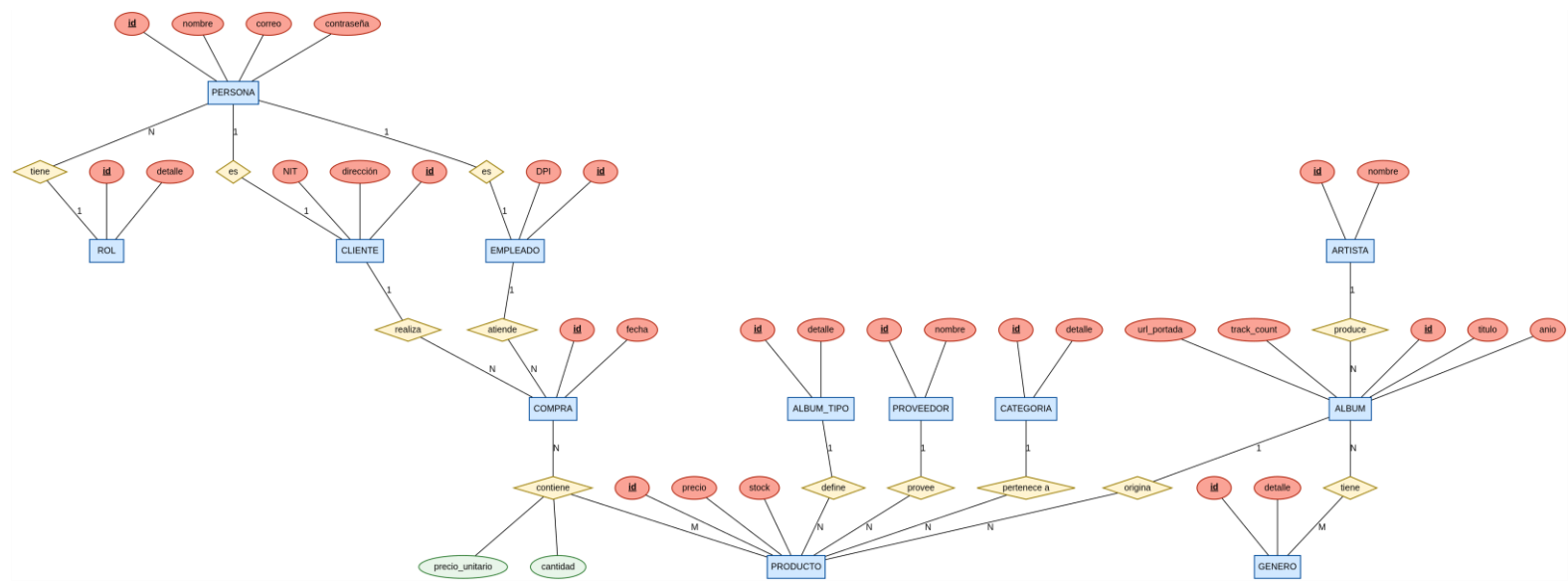
- Id -> id_persona, DPI
- **Compra**
 - Id -> id_cliente, id_empleado, fecha
- **Detalle-venta**
 - (id_compra, id_producto) -> Cantidad, precio_unitario

El diseño de la base de datos se encuentra normalizado en 3FN. Se eliminaron las dependencias parciales al asegurar que el precio_unitario sea un registro histórico dependiente de la llave compuesta en la tabla de cruce. Asimismo, se descartaron las dependencias transitivas al separar los datos personales en la entidad Persona y tratar el subtotal como un atributo derivado calculado en la capa de aplicación. Esta arquitectura garantiza la integridad referencial y optimiza el rendimiento mediante una segmentación lógica de entidades.

Relaciones y Cardinalidades

Entidad 1	Relación	Entidad 2	Cardinalidad
Producto	Pertenece a	Categoría	N:1
Proveedor	Provee	Producto	1:N
Cliente	Realiza	Compra	1:N
Empleado	Atiende	Compra	1:N
Empleado	Es	Persona	1:1
Cliente	Es	Persona	1:1
Persona	tiene	Rol	N:1
Compra	tiene	Producto	N:M
Artista	produce	Album	1:N
Album	Tiene	Genero	N:M
Album	Origina	Producto	1:N
Album_tipo	define	Producto	1:N

Diagrama Entidad Relación (DER)



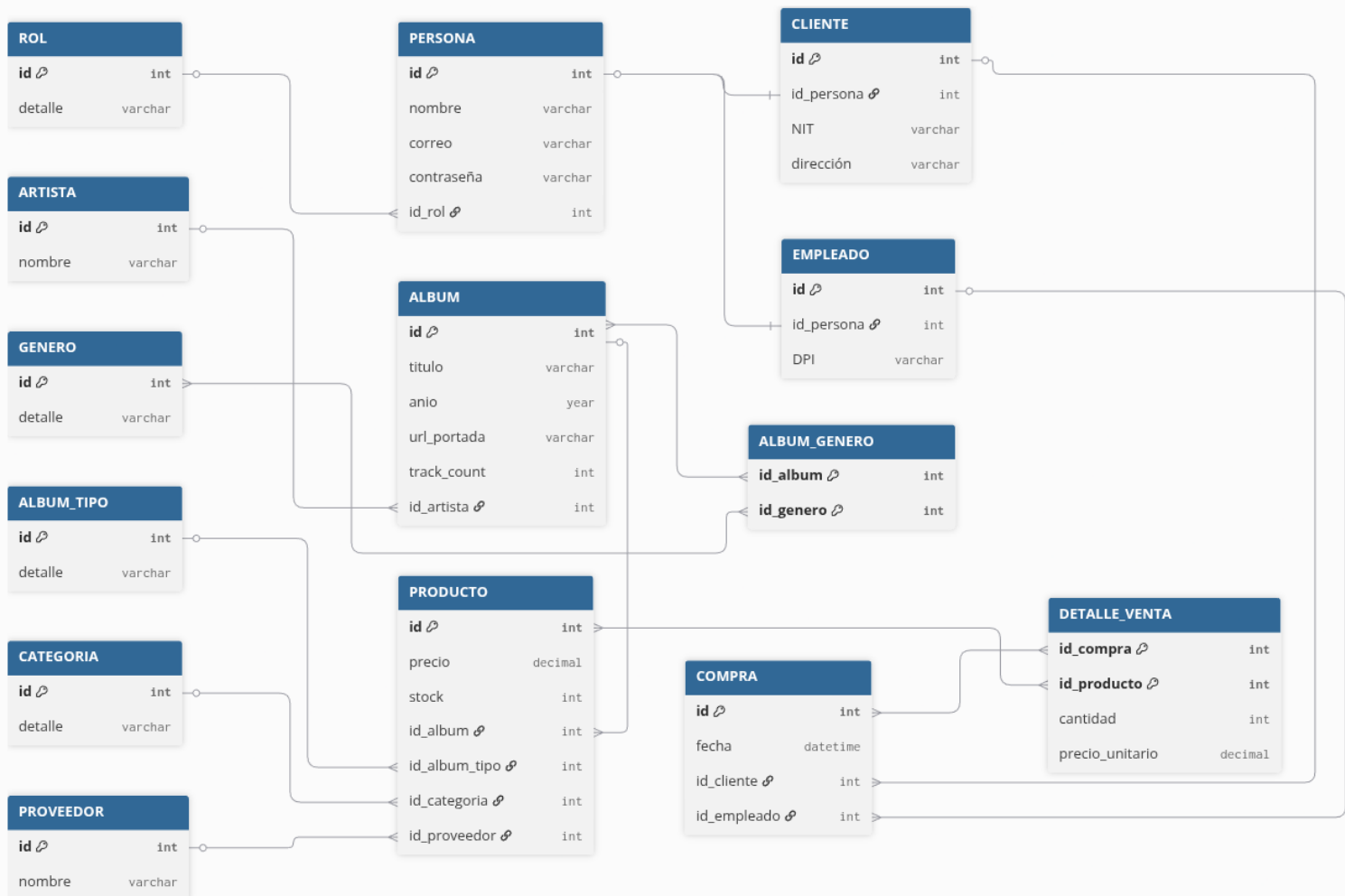
Modelo Relacional

Formato texto

- **ROL** (**ID**, detalle)
- **PERSONA** (**ID**, nombre, correo, contraseña, **id_rol** → **ROL**)
- **CLIENTE** (**ID**, **id_persona** → **PERSONA**, NIT, dirección)
- **EMPLEADO** (**ID**, **id_persona** → **PERSONA**, DPI)
- **CATEGORIA** (**ID**, detalle)
- **PROVEEDOR** (**ID**, nombre)
- **PRODUCTO** (**ID**, precio, stock, **id_album** → **ALBUM**, **id_album_tipo** → **ALBUM_TIPO**, **id_categoria** → **CATEGORIA**, **id_proveedor** → **PROVEEDOR**)
- **COMPRA** (**ID**, fecha, **id_cliente** → **CLIENTE**, **id_empleado** → **EMPLEADO**)
- **DETALLE_VENTA** (**id_compra** → **COMPRA**, **id_producto** → **PRODUCTO**, cantidad, precio_unitario)

- **ARTISTA** (**ID**, nombre)
- **GENERO** (**ID**, detalle)
- **ALBUM_TIPO** (**ID**, detalle)
- **ALBUM** (**ID**, titulo, anio, portada, tracks, **id_artista** → **ARTISTA**)
- **ALBUM_GENERO** (**id_album** → **ALBUM**, **id_genero** → **GENERO**)

Diagrama



Anexos

Repositorio de github: <https://github.com/Diego-glitch-cloud/Proyecto2-DB.git>